

ステップ1: ToDoタスクをlocalStorageに保存する

前の「やること動的サイト」では、ブラウザをリロードすると追加したタスクがすべて消えてしまいました。このステップでは、`localStorage` を利用して、追加したタスクをブラウザに保存する方法を学びます。これにより、ページを再読み込みしてもタスクが保持されるようになります。

1. このステップのゴール

- タスクが追加されるたびに、現在のすべてのタスク情報を `localStorage` に保存する機能を実装する。
- `localStorage` にデータが正しく保存されていることを、ブラウザの開発者ツールを使って確認できるようになる。

2. 事前知識の確認

このステップを進めるにあたり、以下の知識が必要となります。

- `localStorage` の基本 : 前の資料 `localStorageとは.md` を読んで、`localStorage.setItem()`、`JSON.stringify()` の役割を理解しておきましょう。
- JavaScriptの配列とオブジェクト : タスクはオブジェクトとして表現し、それらを配列で管理します。
- DOM操作とイベント処理 : 「やること動的サイト」で学んだ、要素の取得やイベントリスナーの設定方法。

3. 具体的な手順: `script.js` の変更

「やること動的サイト」の `script.js` をベースに、タスクを `localStorage` に保存する機能を追加します。

主な変更点は、タスクを追加する `addTask` 関数と、タスク情報を `localStorage` に保存するための新しい関数 `saveTasks` の作成です。

3.1. タスクを保存するキーの決定

まず、`localStorage` にタスクを保存する際のキー（名前）を決めます。ここでは `'todoTasks'` というキー名を使用することにします。

3.2. `saveTasks` 関数の作成

現在のタスクリスト（`tasks` 配列）を `localStorage` に保存する `saveTasks` 関数を新しく作成します。

```
// script.js の適切な場所（例：tasks 配列の定義後）

// localStorageにタスクを保存するキー
const STORAGE_KEY = 'todoTasks';

// タスクをlocalStorageに保存する関数
function saveTasks() {
  // tasks 配列をJSON文字列に変換してlocalStorageに保存
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
  console.log('タスクをlocalStorageに保存しました:', tasks);
}
```

コード解説:

- `STORAGE_KEY` : `localStorage` に保存する際のキー名を定数として定義しています。こうすることで、キー名を変更したい場合に一箇所修正するだけで済みます。
- `saveTasks()` : この関数が呼び出されると、グローバル変数 `tasks`（すべてのタスクが格納されている配列）を `JSON.stringify()` を使ってJSON文字列に変換し、`localStorage.setItem()` で指定したキー（`STORAGE_KEY`）を使って保存します。
- `console.log()` : 保存が実行されたことを確認するために、コンソールにメッセージと保存内容を出力しています。

3.3. `addTask` 関数の変更

タスクが新しく追加されたときに `saveTasks` 関数を呼び出すように、`addTask` 関数を修正します。

```
// script.js の addTask 関数内

function addTask() {
  const taskText = taskInput.value.trim();
  if (taskText === '') {
    alert('タスクを入力してください。');
    return;
  }

  const newTask = {
    id: Date.now(), // ユニークなIDとしてタイムスタンプを使用
    text: taskText,
    completed: false
  };

  tasks.push(newTask);
```

```
renderTasks();  
taskInput.value = '';  
  
saveTasks(); // ★変更点: タスク追加後に保存処理を呼び出す  
}
```

コード解説:

- `saveTasks();` : `tasks.push(newTask);` と `renderTasks();` の後に追加しました。これにより、新しいタスクが `tasks` 配列に追加され、画面に表示された直後に、更新された `tasks` 配列全体が `localStorage` に保存されます。

3.4. データフローの図解 (イメージ)

タスクが追加されて保存されるまでのデータの流れを簡単に示すと以下のようになります。

```
graph TD  
  A[ユーザーがタスクを入力し追加ボタンを押す] --> B[addTask関数実行];  
  B --> C[tasks配列に新しいタスクオブジェクトを追加];  
  C --> D[renderTasks関数実行];  
  D --> E[画面のリストを更新];  
  C --> F[saveTasks関数実行];  
  F --> G[tasks配列をJSON文字列に変換];  
  G --> H[localStorageに保存 (キー: todoTasks)];
```

4. 演習課題

1. 演習用ベースコード フォルダ内の `script.js` を開き、上記の「3.2. `saveTasks` 関数の作成」と「3.3. `addTask` 関数の変更」で示された変更を加えてください。
 - `STORAGE_KEY` 定数を定義しましょう。
 - `saveTasks` 関数を新しく作成しましょう。
 - `addTask` 関数の末尾で `saveTasks()` を呼び出すように変更しましょう。
2. `index.html` をブラウザで開き、いくつかのタスクを追加してみてください。

5. 動作確認の方法

タスクが正しく `localStorage` に保存されているかを確認するには、ブラウザの開発者ツールを使用します。

1. 開発者ツールを開く : Webページ上で右クリックし、「検証」または「要素を調査」などを選択するか、`F12` キーを押します。

2. 「Application」タブ (Chrome/Edge) または「ストレージ」タブ (Firefox) を選択 : タブの中に「ローカルストレージ」または「Local Storage」という項目があります。
3. ドメインを選択 : 現在開いている `index.html` のドメイン (通常は `file://` から始まります) を選択します。
4. キーと値の確認 : `todoTasks` というキーでデータが保存されているか確認します。値の欄には、タスクの配列がJSON文字列として表示されているはずです。

例えば、タスクを2つ追加した場合、以下のようなデータが確認できるでしょう (IDは異なります)。

```
Key: todoTasks
Value: [{"id":1678886400001,"text":"最初のタスク","completed":false}, {"id":1678886405002,"text":"次のタスク","completed":false}]
```

期待される動作:

- タスクを追加するたびに、開発者ツールの `localStorage` の `todoTasks` の値が更新される。
- ページをリロードすると、現時点ではまだタスクは画面から消えます (読み込み処理は次のステップで実装します) が、`localStorage` にはデータが残っていることを確認してください。

6. Q&A

Q1: なぜ `JSON.stringify()` が必要なのですか？

A1: `localStorage` は、キーと値のペアを保存するストレージですが、保存できる値は文字列型のみです。JavaScriptの配列やオブジェクトをそのまま保存しようとすると、意図した通りに保存されません (例: `[object Object]` のような文字列として保存されてしまう)。

`JSON.stringify()` を使うことで、配列やオブジェクトをJSON形式の文字列に正しく変換でき、`localStorage` に適切に保存できます。

Q2: `STORAGE_KEY` を定数にするメリットは何ですか？

A2: `localStorage` からデータを読み書きする際には、同じキー名を正確に使用する必要があります。キー名を直接文字列で記述すると、タイプミスをしたり、後でキー名を変更したくなった場合に複数箇所を修正する必要が出てきたりします。`STORAGE_KEY` のように定数として定義しておけば、キー名は一箇所で管理され、変更も容易になり、タイプミスによるバグも防ぎやすくなります。

Q3: `id: Date.now()` で生成されるIDは本当にユニーク (一意) ですか？

A3: `Date.now()` は現在のタイムスタンプ（ミリ秒単位）を返すため、通常は異なる時間にタスクを追加すれば異なるIDが生成されます。しかし、非常に短い時間（1ミリ秒以内）に連続してタスクを追加するような特殊なケースや、システム時刻が変更された場合などには、IDが重複する可能性も理論上は存在します。より堅牢なアプリケーションでは、UUID（Universally Unique Identifier）のような、より確実に一意なIDを生成するライブラリを使用することもあります。この演習では簡便さと学習のしやすさを優先して `Date.now()` を使用しています。

7. より詳しいエラーハンドリングについて (コラム)

このステップのコードでは、エラーハンドリングを簡略化しています。しかし、実際のアプリケーションでは、`localStorage.setItem()` が失敗するケースも考慮する必要があります。主な原因としては、以下のものがあります。

- **クォータ超過 (`QuotaExceededError`)**: `localStorage` に保存できるデータ量には上限があります（ブラウザによって異なりますが、通常はドメインごとに5MB〜10MB程度）。この上限を超えてデータを保存しようすると、`QuotaExceededError` が発生します。
- **プライベートブラウジングモードなど**: ブラウザの設定やモードによっては、`localStorage` への書き込みが制限される場合があります。

これらのエラーを適切に処理するためには、`try...catch` 文を使用します。

```
// saveTasks 関数のエラーハンドリング例
function saveTasks() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
    console.log('タスクをlocalStorageに保存しました:', tasks);
  } catch (e) {
    console.error('localStorageへの保存に失敗しました:', e);
    // ユーザーにエラーを通知する処理などをここに追加できます
    if (e.name === 'QuotaExceededError') {
      alert('タスクを保存できませんでした。ストレージの空き容量が不足しています。');
    }
  }
}
```

この演習では、まずは基本的な動作を理解することを優先し、詳細なエラーハンドリングは省略していますが、より実践的な開発ではこのような考慮が重要になります。

よくあるエラーと対処法:

- **`localStorage` に何も保存されない**: `saveTasks()` 関数が正しく呼び出されているか、`STORAGE_KEY` の名前が正しいか確認してください。`console.log` を `saveTasks` 関数内や `addTask` 関数内に追加して、処理の流れを確認しましょう。

- **保存される値が** `[object Object]` **のようになる** : `JSON.stringify()` を忘れずに使っているか確認してください。

次のステップでは、保存されたタスクをページ読み込み時に復元する機能を実装します。